UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN Y CIENCIAS DE LA COMPUTACIÓN
CURSO: AUTÓMATAS Y LENGUAJE FORMALES



PROYECTO FINAL

NOMBRE: DAVID JOSUE VALDEZ RIVERA
LUIS ENRIQUE CHOCHO PORTUGUEZ
EDWIN DAVID ESCOBAR ARANA
JUAN LUIS MOLINA ORELLANA
CARNE :090-24-4642
2990-19-17704
5990-22-6193
0905-24-8364

0ER-AFD

Convertidor de expresiones regulares a autómatas finitos
Documentación técnica
Reporte de algoritmos, pruebas y manual de usuario

Proyecto del curso de Autómatas y Lenguajes Formales
René Ornélyz
Septiembre de 2026

Contenido

1. Descripción, objetivos y alcance
2. Diseño general y estructura del código
3. Lenguaje de expresiones regulares admitido
4. Algoritmos de conversión ER → AFND → AFD
5. Evaluación de cadenas y formatos de salida
6. Instalación y manual de usuario
7. Casos de prueba y evidencias
8. Verificación, limitaciones y cierre

Propósito del documento

Esta documentación describe la implementación del proyecto entregado: un analizador propio de expresiones regulares, la construcción de un AFND mediante Thompson, su determinización por subconjuntos y la simulación de cadenas con el AFD resultante. También explica cómo ejecutar el programa y presenta capturas tomadas de la interfaz web local.

El modo obligatorio de línea de comandos se conserva. La aplicación web y la interfaz Tkinter son complementos que reutilizan el mismo motor. Graphviz es opcional y no participa en el reconocimiento.

1. Descripción y objetivos

Descripción

ER-AFD recibe una expresión regular y una cadena de entrada. Construye primero un autómata finito no determinista con transiciones ε y luego un autómata finito determinista equivalente. El AFD procesa la cadena completa y determina si termina en un estado de aceptación.

El proyecto implementa el reconocimiento sin utilizar bibliotecas de expresiones regulares para decidir la aceptación. El analizador y los dos algoritmos de conversión son código propio del paquete `erafd`.

Objetivo principal

Aplicar los conceptos de conversión de expresiones regulares a autómatas finitos para reconocer patrones de manera explícita y verificable.

Objetivos específicos

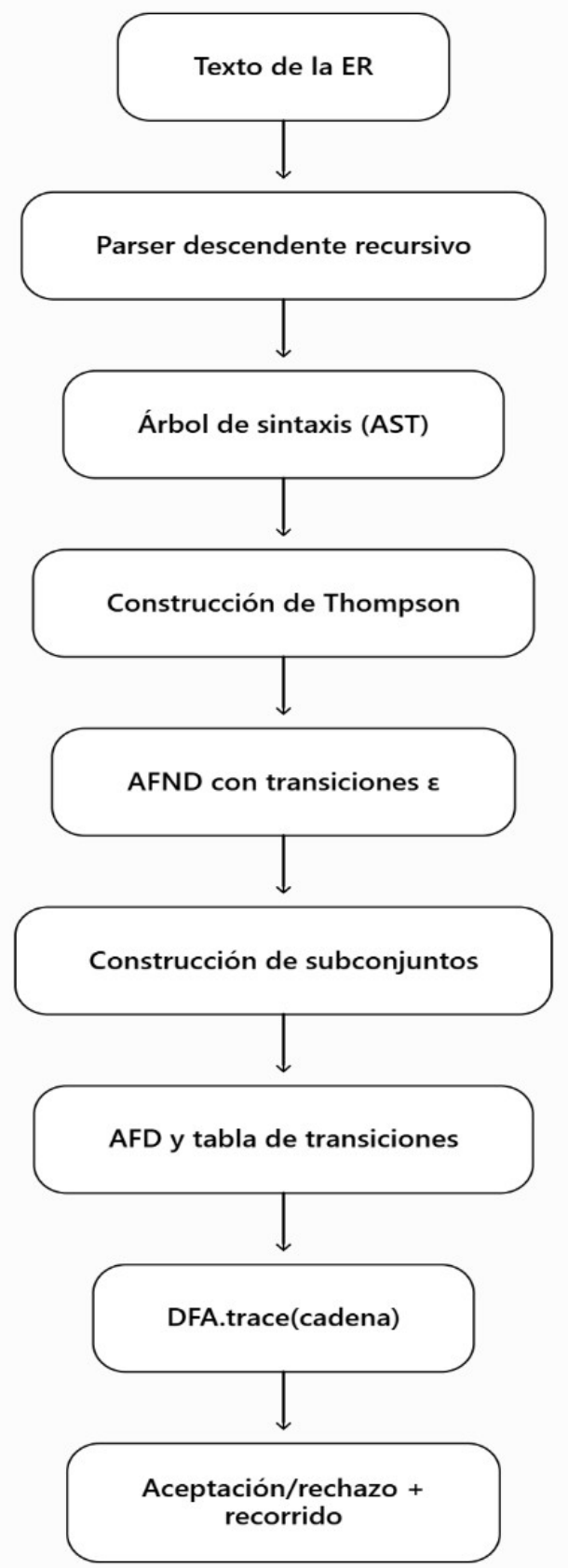
- Transformar una ER en un AFND y después en un AFD.
- Mostrar estados, alfabeto, transiciones y estados de aceptación.
- Evaluar cadenas y mostrar paso a paso el recorrido del AFD.
- Probar la lógica con pruebas automatizadas y casos reproducibles.

Alcance funcional

- CLI con la forma `er-afd <ER> <cadena>`.
- Interfaz web local con recorrido, tablas AFND/AFD y diagramas.
- Interfaz gráfica complementaria basada en Tkinter.
- Exportación de diagramas DOT y, con Graphviz, imágenes PNG/SVG.

El proyecto analiza una ER por ejecución; no tokeniza una lista de ER ni procesa directamente archivos fuente. El alfabeto y la cadena se limitan a ASCII, según el motor implementado.

2. Diseño y estructura del código



Archivo	Responsabilidad
erafd/core.py	Parser, estructuras de autómatas, conversión, simulación y formato de tablas.
erafd/__main__.py	Interfaz de línea de comandos y códigos de salida.
erafd/web/server.py	Servidor HTTP local, validación de solicitudes y respuesta JSON de análisis.
erafd/web/static/	HTML, CSS y JavaScript de la aplicación web.
erafd/diagram.py	Generación de DOT y conversión opcional a PNG con Graphviz.
erafd/gui.py	Interfaz gráfica complementaria basada en Tkinter.
tests/	Pruebas del motor, la CLI, los diagramas y la API web.
Makefile, iniciar.*	Tareas comunes y lanzadores para Linux y Windows.

Representación de datos

- Node representa operadores y átomos en el árbol de sintaxis.
- Edge contiene origen, destino, máscara de caracteres y etiqueta.
- NFA almacena estados, inicio, final, transiciones y símbolos.
- DFA guarda subconjuntos de estados AFND, finales y función delta.

En las tablas, los estados del AFND se escriben qi y los del AFD Qi. Ambos autómatas comienzan en el estado 0. Una transición ε no consume un carácter de la cadena.

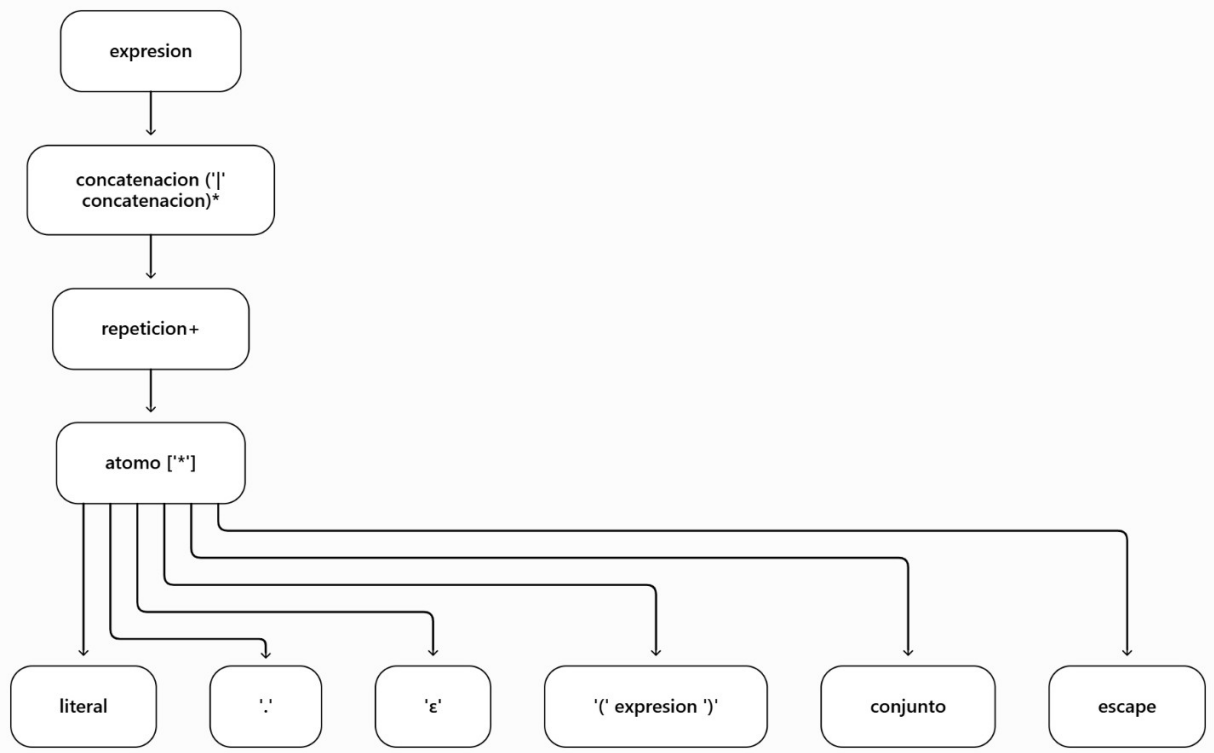
3. Expresiones regulares admitidas

Notación	Significado	Ejemplo
ab	Concatenación implícita: primero a, después b.	ab acepta «ab».
|	Unión o alternativa.	a|b acepta «a» o «b».
*	Cerradura de Kleene: cero o más repeticiones del átomo anterior.	a* acepta ε, «a», «aa», etc.
(...)	Agrupación y control de precedencia.	(a|b)*abb.
.	Cualquier carácter ASCII de 0 a 127.	a.b acepta «a0b» y «a b».
ε	Cadena vacía; no consume entrada.	ε acepta la cadena vacía.
[abc]	Un carácter de los enumerados.	[abc] acepta «a», «b» o «c».
[^abc]	Un carácter ASCII que no sea a, b ni c.	[^ab].
[a-z]	Un carácter dentro del rango, inclusive.	[a-z] acepta una minúscula ASCII.

Secuencias de escape

Escape	Conjunto o valor representado
\N	Dígitos ASCII, equivalente a [0-9].
\A	Letras mayúsculas ASCII, equivalente a [A-Z].
\a	Letras minúsculas ASCII, equivalente a [a-z].
\z	Letras ASCII, equivalente a [a-zA-Z].
\hHH	Carácter indicado por dos dígitos hexadecimales entre 00 y 7F; por ejemplo, \h20 es espacio y \h41 es «A».
\X	El carácter siguiente de forma literal, incluso si es un operador; por ejemplo * o \p.

Gramática simplificada y precedencia



La precedencia, de mayor a menor, es: repetición *, concatenación y unión |. El parser rechaza expresiones vacías, paréntesis o clases sin cerrar, operandos faltantes y dos * consecutivos. Para representar la cadena vacía se usa ε.

4. Algoritmos de conversión

4.1 Análisis descendente recursivo

Parser consume la expresión de izquierda a derecha. Los métodos union(), concat(), repeat() y atom() reflejan la gramática y su precedencia. Cada método construye un nodo Node; por ejemplo, a|bc* queda estructurado como una unión entre a y la concatenación de b con c*.

Las clases se representan internamente como máscaras de bits: el bit número n indica si el código ASCII n pertenece al conjunto. Esto permite probar una transición de símbolo mediante una operación de bits, sin llamar a un motor de regex.

4.2 Construcción de Thompson: ER → AFND

Cada subexpresión produce un fragmento con un estado inicial y uno final. Los casos de construcción son:

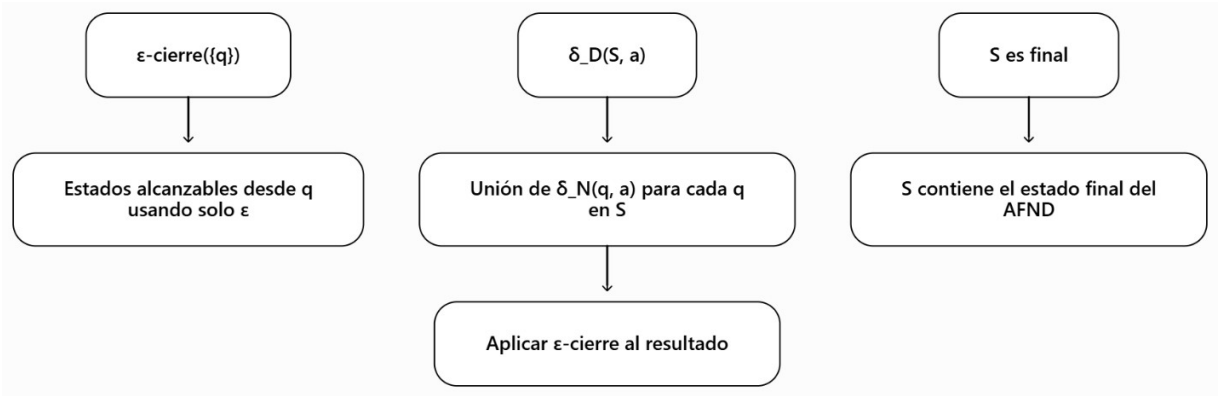
Construcción	Fragmento creado
Símbolo o clase	Dos estados unidos por una transición etiquetada con el símbolo o máscara.
ε	Dos estados unidos por una transición vacía (sin consumir entrada).
Concatenación A B	El final de A se conecta con el inicio de B mediante ε.
Unión A | B	Se añade un inicio común con ramas ε hacia A y B; ambos finales se conectan por ε a un final común.
Cerradura A*	Un nuevo inicio permite saltar A o entrar en A; el final de A permite repetir o salir al final común.

La implementación usa Thompson.fragment() recursivamente. Las transiciones ε se guardan con máscara None; las transiciones que consumen entrada contienen la máscara del símbolo. Después de generar el fragmento completo, los estados se renumeran comenzando desde q0.

4.3 Construcción de subconjuntos: AFND → AFD

Un estado del AFD representa un conjunto de estados posibles del AFND. La función subset_construction() utiliza una búsqueda en anchura para descubrir esos conjuntos y construir la tabla delta.

1. **Cierre ε:** dado un conjunto, agrega todos los estados alcanzables siguiendo cero o más transiciones ε.
2. **Estado inicial:** Q0 es el cierre ε del estado inicial q0.
3. **Movimiento:** para cada conjunto y símbolo, reúne los destinos de las transiciones que consumen ese símbolo.
4. **Cierre del destino:** calcula el cierre ε del conjunto alcanzado; ese conjunto es el estado destino.
5. **Registro:** si el conjunto aún no existe, se crea un nuevo estado AFD y se procesa después.
6. **Aceptación:** un estado AFD es final cuando el conjunto que representa contiene el estado final del AFND.



El conjunto vacío también puede aparecer como estado destino cuando no existe movimiento válido para un símbolo que sí pertenece al alfabeto. El límite de seguridad de la implementación es de 2048 estados AFD; al excederlo, la conversión informa un error en lugar de continuar creciendo.

Presentación de las tablas

Las tablas muestran una fila por transición con el formato estado | símbolo | destino. Las clases equivalentes del AFD pueden agruparse cuando producen el mismo comportamiento. Los nombres de los estados se asignan por el orden en que se descubren; por ello una numeración distinta puede representar un AFD equivalente.

5. Evaluación y salidas

Simulación de una cadena

DFA.trace() comienza en Q0 y lee un carácter a la vez. En cada paso consulta la transición correspondiente en la fila del estado actual. Si no existe transición para el carácter, el recorrido termina y la cadena se rechaza. Si se consume toda la entrada, se acepta únicamente cuando el estado final alcanzado pertenece al conjunto de aceptación.

La comparación es de cadena completa, no de subcadena. Una entrada vacía no genera pasos: se acepta si Q0 es final. El motor rechaza caracteres de entrada no ASCII.

Ejemplo de recorrido

ER (a|b)*abb, cadena aabb:

Q0|a|Q1
Q1|a|Q1
Q1|b|Q3
Q3|b|Q4
Cadena aceptada

El estado final Q4 se alcanza después de consumir toda la cadena. La misma expresión rechaza abaa, porque el recorrido no termina en un estado de aceptación.

Salida de la CLI

Código	Significado del proyecto
1	Cadena aceptada.
0	Cadena rechazada.
2	Error de argumentos, ER inválida o entrada no admitida.

El código 1 es inusual para éxito en una CLI convencional y algunos shells lo muestran como estado distinto de cero. En este proyecto es intencional: así lo define el contrato de evaluación.

Respuesta de la aplicación web

La API entrega resultado booleano, recorrido, ruta de estados, métricas, tablas AFND/AFD, subconjuntos y datos del diagrama en JSON. La interfaz presenta esa información en pestañas. El servidor escucha en 127.0.0.1 y valida host, origen, tipo de contenido y tamaño de entrada.

6. Manual de instalación y ejecución

Requisitos

- Python 3.9 o posterior.
- Navegador web para la interfaz web.
- Graphviz es opcional; se requiere solo para los diagramas renderizados.

Windows: iniciar la aplicación web

1. Instalar Python desde python.org y activar la opción de añadirlo al PATH.
2. Abrir la carpeta del proyecto en el Explorador de archivos.
3. Ejecutar iniciar.bat, o abrir PowerShell en esa carpeta y escribir:

```
py -3 -m erafd.web.server
```

El navegador abre http://127.0.0.1:8765/. Para usar otro puerto:

```
.\iniciar.bat --port 8766
```

Linux: iniciar la aplicación web

Desde una terminal situada en la carpeta del proyecto:

```
./iniciar.sh
# O indicar otro puerto:
./iniciar.sh --port 8766
```

Para detener el servidor, regresar a la terminal y presionar Ctrl+C.

Usar la CLI

La forma general es:

```
py -3 -m erafd "(a|b)*abb" "aabb"
```

En Linux también puede utilizarse:

```
./er-afd '(a|b)*abb' 'aabb'
```

Se recomienda encerrar la ER y la cadena entre comillas para que el shell no interprete los operadores o espacios. La cadena vacía puede pasarse como segundo argumento vacío, por ejemplo py -3 -m erafd "ε" "".

Usar la interfaz

1. Escribir la expresión regular y la cadena de prueba.
2. Seleccionar **Convertir y evaluar** o usar Ctrl+Enter.
3. Consultar el resultado y el recorrido; luego abrir las pestañas de tablas o diagramas.
4. Para empezar de nuevo, elegir **Limpiar**.

Para ejecutar la interfaz Tkinter: py -3 -m erafd.gui. Esta interfaz es opcional y puede requerir que Tkinter esté disponible en el sistema.

7. Casos de prueba y evidencias

Las siguientes capturas se generaron al ejecutar la aplicación web local. Los ejemplos usan la ER (a|b)*abb, que acepta cadenas sobre el alfabeto {a,b} que terminan en abb.

Caso	ER	Cadena	Resultado esperado	Qué verifica
CP-01	(a|b)*abb	aabb	Aceptada (código 1)	Un prefijo válido seguido por el sufijo requerido.
CP-02	(a|b)*abb	abaa	Rechazada (código 0)	La cadena usa símbolos permitidos, pero no termina en abb.
CP-03	ε	Vacía	Aceptada (código 1)	Evaluación de la cadena vacía y cierre ε inicial.
CP-04	(a|	a	Error (código CLI 2)	Detección de una ER sintácticamente incompleta.

CP-01 a CP-03 tienen evidencia visual en las páginas siguientes. CP-04 está cubierto por las pruebas automatizadas de sintaxis y de la API.

CP-01 · Cadena aceptada

Entrada: ER (a|b)*abb · cadena aabb.

Resultado observado: aceptada. El recorrido termina en el estado final Q4.

ER a AFD

Convertidor de expresiones regulares

Expresión regular

```
(a|b)*abb
```

Cadena para evaluar

```
aabb
```

Convertir y evaluar | Limpiar

Ejemplo: `(a|b)*abb` con la cadena `aabb`. Deja la cadena vacía para probar ε.

Cadena aceptada

AFND: 14 estados | AFD: 5 estados | Código: 1

Recorrido | Tabla AFND | Tabla AFD | Diagramas

Evaluación de la cadena

```
Q0|a|Q1
Q1|a|Q1
Q1|b|Q3
Q3|b|Q4
Cadena aceptada
```

Figura 1. Ejecución web del caso CP-01, con recorrido completo y código de retorno 1.

CP-02 · Cadena rechazada

Entrada: ER (a|b)*abb · cadena abaa.

Resultado observado: rechazada. La cadena no termina con el sufijo abb.

ER a AFD

Convertidor de expresiones regulares

Expresión regular

```
(a|b)*abb
```

Cadena para evaluar

```
abaa
```

Convertir y evaluar | Limpiar

Ejemplo: `(a|b)*abb` con la cadena `aabb`. Deja la cadena vacía para probar ε.

Cadena rechazada

AFND: 14 estados | AFD: 5 estados | Código: 0

Recorrido Tabla AFND Tabla AFD Diagramas

Evaluación de la cadena

```
Q0|a|Q1
Q1|b|Q3
Q3|a|Q1
Q1|a|Q1
Cadena rechazada
```

Figura 2. Ejecución web del caso CP-02, con resultado de rechazo y código de retorno 0.

CP-03 · Cadena vacía

Entrada: ER ε · cadena vacía.

Resultado observado: aceptada sin consumir símbolos; Q0 es un estado final.

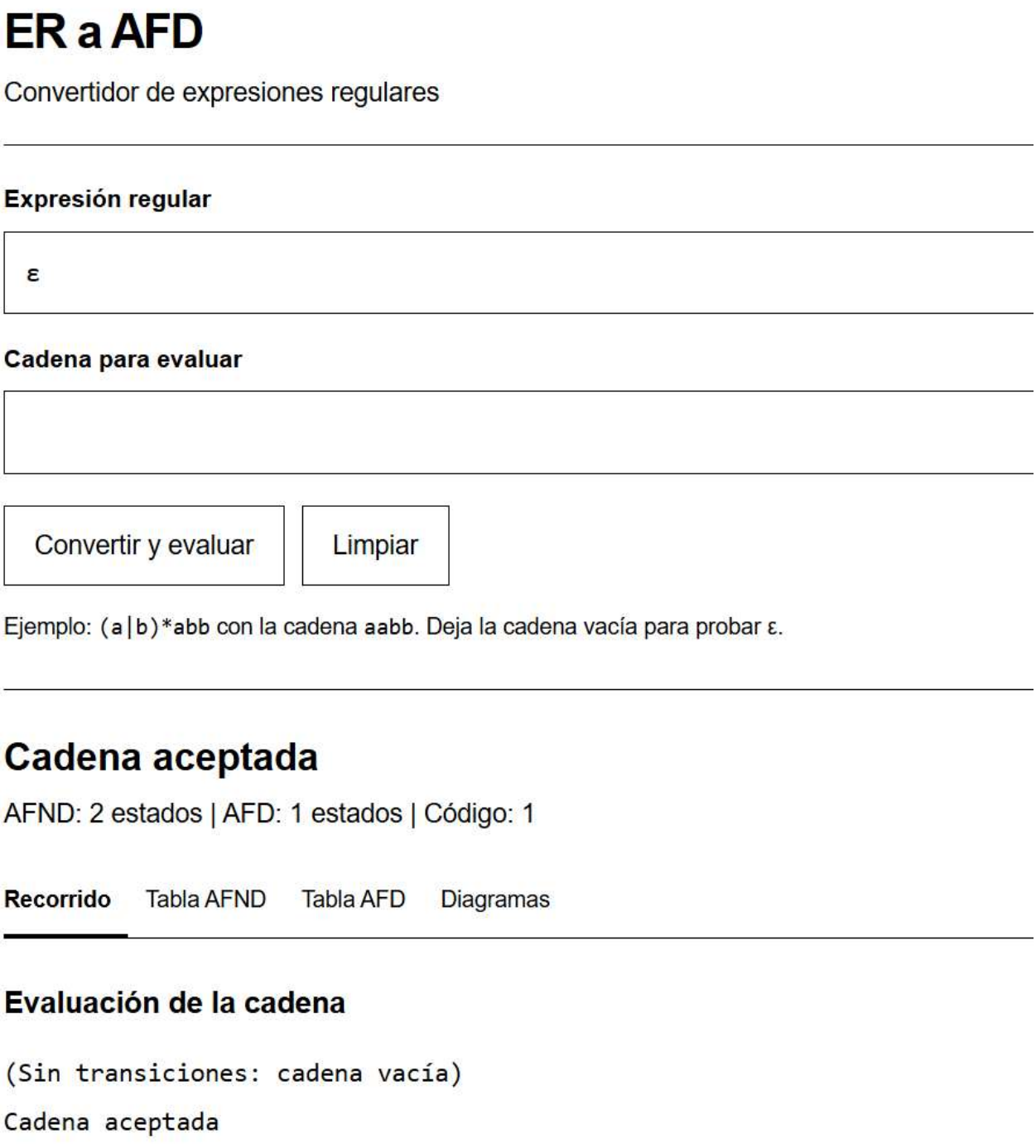


Figura 3. Ejecución web del caso CP-03; no hay transiciones de entrada y la cadena vacía se acepta.

8. Pruebas automatizadas y limitaciones

Pruebas ejecutadas

Comando usado desde la raíz del proyecto:

```
py -3 -m unittest discover -s tests -v
```

Resultado comprobado: 124 pruebas ejecutadas; 122 aprobadas y 2 omitidas. Las dos pruebas omitidas son las que requieren Graphviz instalado para generar imágenes PNG. Las pruebas restantes validan conversiones, precedencia, operadores, clases, escapes, errores de sintaxis, recorrido, CLI, exportación DOT, API web, límites y recursos estáticos.

Restricciones conocidas

- El alfabeto del motor es ASCII (0–127); no acepta letras acentuadas en la cadena.
- La interfaz web limita la ER a 320 caracteres y la cadena a 4000; el AFD tiene un límite de 2048 estados.
- El parser implementa los operadores documentados; no implementa `+`, `?`, cuantificadores `{n}` ni referencias hacia atrás.
- Graphviz solo es necesario para los diagramas renderizados. La conversión y las tablas funcionan sin él.
- El servidor web está pensado para uso local y escucha únicamente en loopback.

Conclusión

El programa hace explícita la cadena de transformación académica ER → AFND → AFD. La tabla y el recorrido permiten inspeccionar cómo cada carácter modifica el estado del autómata. Las pruebas automatizadas respaldan los casos principales y la interfaz ofrece una forma visual de explorar los resultados sin reemplazar la CLI requerida.

Archivos fuente de referencia: `erafd/core.py`, `erafd/__main__.py`, `erafd/web/server.py`, `erafd/diagram.py` y `tests/`.